Aurik Equipment Monitoring

Project guide: what was built, how it works,
and how to run it and see the output

Project	Industrial equipment monitoring backend service (Aurik Tech Lead assessment)
Version	1.5.0 (rules version 1.3)
Stack	Backend: Python, FastAPI, Pydantic, SQLite · Frontend: React, TypeScript, Vite
Date	October 2026

Contents

1. The project at a glance
2. How the system works
3. What was built, part by part
4. How messy vendor data is handled
5. What we did during development
6. Project structure
7. How to run the project
8. How to see the output
9. Troubleshooting
10. Known limitations

1. The project at a glance

The problem. A factory receives machine data from several external vendors. Each vendor uses its own format: different field names, time formats, units, alert codes and severity scales. The data is also messy: records arrive twice, out of order, with missing or invalid values, and different vendors sometimes disagree about the same machine.

What the system does. It accepts that data through an API, stores every payload exactly as received, converts it into one common (canonical) format in the background, and then works out, for every machine, **whether it needs attention, how urgently, and why**. Plant and line summaries and a web dashboard sit on top.

4
data sources: PulseForge, ThermexWatch, MaintaFlow + internal work orders

8
machines across 2 plants and 5 production lines

24
documented API endpoints (OpenAPI)

169
automated tests, all passing (153 backend + 16 frontend), plus a 55-check end-to-end run on Docker

The four data sources

Source	What it sends	Format quirks the system has to handle
PulseForge	Sensor events (vibration, temperature, machine state)	ISO-8601 time; severity as words (low…critical); sensor health 0–1
ThermexWatch	Sensor readings and alerts	Time in epoch milliseconds; temperature in °F; vibration in g; level 1–5; short line codes ("A")
MaintaFlow	Inspections, maintenance updates, operator notes, calibration	Time as `2026/04/18 08:14:05` with no time zone; free-text notes; confidence as words
CMMS work orders	Internal maintenance work orders (open / closed, post-maintenance check)	A work order changes state over time, so the same id arrives more than once

What a downstream team gets for each machine

Derived status (healthy → critical), whether it needs attention, an attention level 0–4, reason codes with a plain-English explanation for each, the latest relevant signals, how fresh the data is, and links back to the exact source records. This is the "machine operational attention view" the brief asks for.

2. How the system works

1. Ingest API
Checks the request envelope, stores every record raw, removes duplicates, answers `202` immediately.

→

2. Raw store + queue
`raw_records`: append-only copy of what was received; also the work queue.

→

3. Worker
Picks up pending records, converts them to the canonical format, retries on failure.

→

4. Rules engine
Recomputes each affected machine from all its events: status, level, reasons.

→

5. Read APIs + dashboard
Machine view, summaries, batch status, dead letters, traceability.

Step by step

1. **Accept.** A vendor posts a batch to `POST /v1/ingest/{source}`. The envelope is validated straight away (valid JSON, correct vendor, non-empty list of records). Each record is stored as received, duplicates are counted, and the API returns `202 Accepted` with a `batch_id`. Nothing slow happens on this request.
2. **Process in the background.** A worker (a thread in the API process, or a separate container) claims pending records with a time-limited lease, normalizes them, and saves a canonical event. Bad records go to the dead-letter list with an error code; temporary failures are retried with exponential backoff.
3. **Derive.** Every machine that received new data is recomputed from *all* of its events using deterministic rules. Because the result is recomputed from scratch and uses event time (not arrival time), the order in which data arrives cannot change the answer.
4. **Serve.** Read APIs return the machine view and summaries. Freshness ("processing status") is calculated when you ask, so it is always current.

Key design decisions

Decision	Why
Store raw data first, append-only	Every answer can be traced back to the exact bytes received; mistakes in normalization can be fixed by replaying.
Database table as the job queue	One moving part instead of Redis/Kafka; safe with several workers; easy to test. Swap for Postgres or SQS at scale.
Reject vs. warn	A missing machine id or unreadable time rejects the record. A bad optional value (e.g. `-999`) is nulled with a warning, so one bad reading does not hide a real alert.
Ordered revisions	When a vendor re-sends a record, every version is kept, but only the newest one counts: newest vendor batch time, then the record's own time, then arrival order. A late-arriving older version can never overwrite a newer one.
Scoped repairs	A verified work order clears only what it repaired: a power check clears power alerts, not vibration, temperature or inspection findings.
Two freshness policies	Sensor readings expire after 60 minutes (relative to the machine's newest reading). Maintenance findings such as an inspection defect never expire by age; they stay until someone resolves them.
Pessimistic on conflicts	If one vendor says "recovered" and another still reports an alert, the alert wins and the conflict is flagged. Missing a real fault costs more than a false alarm.
Explainable rules, no ML	Every level comes with reason codes, the source record and a sentence explaining it, as the brief prefers.

Backend and frontend separate	The dashboard uses only the public API, so it can never depend on anything downstream teams cannot use.

3. What was built, part by part

A. Ingestion

- One typed endpoint per source: `/v1/ingest/pulseforge`, `/thermexwatch`, `/maintaflow`, `/workorders`.
- **Two-level validation.** Level 1 (immediate, whole request): JSON, content type (`415`), size limits (`413`), record list present and non-empty, every record an object, vendor name matches the endpoint (`422` with the exact field path). Level 2 (per record, in the background): ids, machine, timestamps, enums, units, value ranges.
- **Dry run.** `/v1/ingest/{source}/validate` shows what would happen to every record (valid / warnings / rejected; new / duplicate / revision) without storing anything.
- **Duplicates and idempotency.** Re-sending the same request returns the original batch (`200`). Same record id with the same content counts as a duplicate; same id with new content becomes a new revision (the newest by vendor batch time, then record time, counts; history kept); same content under a new id is caught by a content fingerprint.
- **Security.** Optional API keys (`X-API-Key`), one consistent error format, and a request id on every response.

B. Normalization

Concern	What the system does
Machine resolution	`EQ-1`, `eq_001` → `EQ-001`; line `C` → `LINE-C`; plant and line always taken from the asset reference file.
Timestamps	ISO-8601, epoch milliseconds and `YYYY/MM/DD HH:MM:SS` all converted to UTC.
Units	°F → °C. Vibration in g is kept as g (converting to mm/s would need frequency data, so faking it would be wrong).
Enums	Vendor alert codes, severity words and levels 1–5 mapped to one canonical set (based on `event_type_reference.md`).
Missing / invalid values	Essential field bad → record rejected with a code. Optional value bad → set to null with a warning code.

C. Background processing

Lease-based claiming (safe with several workers), exponential-backoff retries, a dead-letter list after `MAX_ATTEMPTS`, recovery when a worker crashes (its lease expires), and a durable "recompute this machine" marker so state is never left half-updated. Rejected records can be **replayed** after the data or reference is fixed.

D. Derived machine view: attention levels

Level	Label	Derived status	Needs attention
0	none	healthy	no
1	low	watch	no
2	medium	degraded	yes
3	high	at_risk	yes

4	critical	critical	yes

Also: `under_maintenance` (asset reference says so), `offline`, and `unknown` (no data yet). Reasons come from vendor alerts, from comparing readings with each machine's rated limits (temperature, vibration, power), from maintenance findings, and from work orders. Poor signal quality or low technician confidence lowers a reason by one level; two independent sources both reporting "high" raise the level by one.

Freshness (rules 1.2). A sensor reading is a measurement at one moment: a vendor's newest reading is ignored (`STALE_SIGNAL_IGNORED`) when it is more than 60 minutes older than the machine's newest sensor reading from any vendor. A maintenance finding (inspection defect, pending calibration, operator-reported issue) describes a condition that stays true until someone deals with it, so it never expires by age: the reason says "unresolved since …" and how old it is. It is resolved by a newer inspection that passed (`FINDING_RESOLVED_BY_INSPECTION`), a newer calibration that is no longer pending (`FINDING_RESOLVED_BY_CALIBRATION`), completed maintenance, or a verified work order.

Revisions (the same record sent again). Every version is stored unchanged. The one that counts is the newest by vendor batch time (PulseForge `batch_generated_at`, ThermexWatch `response_time_epoch_ms`), then by the record's own time (the only option for MaintaFlow and work orders, which send no batch time), then by arrival. An older version that arrives late is kept in the history as `superseded` / `STALE_REVISION`. Different record ids are separate events, never revisions.

Work orders (rule 8). A work order closed with a *passed* post-maintenance check proves that *what it repaired* works. It resolves only earlier findings in its scope: the categories of its `maintenance_type` (`power_check` → power, `cooling_system` / `heat_shield` → temperature, `sensor_recalibration` → sensor and calibration) plus what triggered it (an inspection finding or an operator report). Resolved findings stay visible at level 0 with the work order named. Only findings dated at or before the close are resolved, so an older repair never clears a newer fault. An open or reopened work order is shown as "acknowledged" but does not lower attention; a failed or pending check raises it; an unknown maintenance type resolves nothing.

Reject or warn? A record is **rejected** (dead letter, replayable) when it cannot be identified or placed in time: missing id, missing or unknown machine, unreadable or missing timestamp, or a time more than 10 minutes after it arrived (`FUTURE_TIMESTAMP`). Everything else is **kept with a warning**: wrong plant or line (corrected from the reference), out-of-range or non-finite numbers (nulled), unknown booleans or enum values, and an alert code that contradicts the level (`ALERT_LEVEL_CONFLICT`, the more severe reading wins). Warnings show in the trace, the dry run and on each signal.

4. How messy vendor data is handled

The assessment supplied edge-case files for each kind of messy data. Each one is covered by an automated test.

Situation	Supplied file	Result
Clean data	`happy_path.json`	Processed asynchronously; machines get status and reasons.
Duplicates	`duplicates.json`	The identical second copy is counted as a duplicate; the copy re-sent under a new id (`TW-8801-R`) is detected by content fingerprint.
Out of order	`out_of_order.json`	A recovery listed before an older alert still wins, because event time decides, not arrival order.
Malformed	`malformed.json` (two sections)	**PulseForge section:** both records rejected: PF-1201 (empty machine id) and PF-1202 (timestamp `18-04-2026 08:28:20` is not ISO-8601). PF-1202 also contains `-999`, `"bad"` and confidence 1.2, but it is rejected for its timestamp before those values matter. **ThermexWatch section:** TW-9911 rejected (no machine id); TW-9912 processed with warnings (`UNMAPPED_ALERT_CODE`, `LINE_MISMATCH_USED_REFERENCE`).
Conflicting	`conflicting_updates.json`	PulseForge says recovered, ThermexWatch says critical, MaintaFlow finds a major defect → critical, flagged `CONFLICTING_SOURCES`.
Partial failure	`retry_cases.json`	Good records are processed, bad ones rejected. A corrected re-send of the same id clears the dead letter.
Late older version	(regression tests)	A record re-sent with an older vendor batch time after a newer one is kept as `superseded` / `STALE_REVISION`; the newer version stays authoritative, in any arrival or processing order.
Old findings, new readings	(regression tests)	An April inspection defect is still reported next to an October reading; an April sensor reading is ignored. See section 8.5.
Work orders	`maintenance_work_orders.csv`	WO-1002 (`power_check`, triggered by an operator report, closed with a passed check) resolves EQ-004's two power alerts and the operator note. Open orders on EQ-002, EQ-007, EQ-008 are shown as acknowledged.
Duplicate vs new	(regression tests)	The same reading re-sent under a new id is a duplicate; a reading that differs in any value (even only signal quality) is a new observation and is kept.

> **Raw data is never changed.** The request body is stored byte-for-byte (`GET /v1/batches/{id}/raw`) and every record keeps all of its fields and values as sent. All 15 supplied asset files are in the repository byte-identical to the original package.

5. What we did during development

The project already had a working backend. The work then went through these stages:

Stage	What changed and why
1. Review and first bug fix	Found that every MaintaFlow record failed on Windows: time-zone data (`tzdata`) was missing. Fixed and added the package.
2. Folder architecture	Split the flat `app/` folder into layers: `api` (HTTP only) → `services` (use cases) → `domain` (pure normalization and rules), plus `schemas`, `db`, `worker`, `core`. File history was preserved with `git mv`.
3. Proper validation	Pydantic envelope models per vendor, typed query and path parameters, response models, one error format, the dry-run endpoint, request ids, media-type and size checks.
4. Dashboard	An operations dashboard: overview, machines, ingest, batches, dead letters, trace.
5. Backend / frontend split	Moved into `backend/` and `frontend/`, each with its own Dockerfile; added a third Docker Compose service (nginx serving the dashboard and proxying the API).
6. React frontend	Rebuilt the dashboard in React + TypeScript (Vite, TanStack Query, React Router), with typed API models and unit tests. Same look and features.
7. Requirements audit	Checked the code against the brief and every supplied file. Fixed: work-order file not used (added as a fourth source + rule 8); raw bytes not kept (now stored); the same record from a Python and a JavaScript client counted as a revision (hashing fixed); wrong fallback severity for `TEMP_CRIT` (fixed); added tests for each.
8. Docker + policy review	The Docker Compose stack was built and run successfully (see section 7). A review then fixed revision handling (a late older version could replace a newer one), anchored sensor relevance to sensor time, stated the age of unresolved findings, added explicit "resolved by" reasons, corrected this guide's malformed-data examples, and added an end-to-end verification script.
9. Requirements audit 2	A full audit against the brief and assets fixed: work orders clearing unrelated findings (now scoped), distinct readings wrongly dropped as duplicates (fingerprint now covers every field), a stalled worker able to overwrite results (lease fencing), future timestamps accepted, unknown booleans and `OK`-with-high-level silently trusted, equal-time readings tie-broken by id, and exact threshold boundaries lost to floating-point error. Added reopened work orders, data-arrival time, warnings in the machine view, 57 regression tests, and an isolated Docker verification.

6. Project structure

```
aurik-monitoring/
├── backend/                      Python service (own Dockerfile, requirements, tests)
│   ├── app/
│   │   ├── main.py               app setup: middleware, CORS, error handlers, routes
│   │   ├── api/routes/           HTTP endpoints: health, ingestion, processing, machines, samples
│   │   ├── schemas/              request envelopes and response models = the API contract
│   │   ├── services/             ingestion, processing (dead letters, trace), read views
│   │   ├── domain/
│   │   │   ├── normalization/    one file per source + shared rules + vocabulary
│   │   │   ├── rules.py          attention rules (pure function, no I/O)
│   │   │   └── vendors.py        registry of the four sources
│   │   ├── worker/               background processing (lease, retry, recompute)
│   │   ├── db/                   schema, connections, reference-data loading
│   │   └── core/                 config, errors, API-key auth, time helpers
│   ├── tests/                    153 tests (9 files)
│   ├── scripts/                  load_seed.py, verify_scenarios.py, export_openapi.py
│   ├── data/                     reference + seed CSVs (as supplied)
│   └── sample_payloads/          vendor JSON samples (as supplied)
├── frontend/                     React + TypeScript dashboard (own Dockerfile + nginx.conf)
│   └── src/                      api/, pages/, components/, charts/, features/, lib/
├── docs/                         architecture, design note, OpenAPI, requirements coverage
├── docker-compose.yml            api + worker + frontend
├── Makefile                      shortcuts (needs make; on Windows use the commands in section 7)
└── README.md
```

Dependency direction: `api → services → domain`. The domain layer never touches HTTP or the database, which is why the normalizers and rules can be tested as plain functions.

7. How to run the project

Prerequisites

- Python 3.12 or newer, Node.js 20 or newer (for the dashboard), Git.
- Optional: Docker Desktop (for option B).

Option A: run locally on Windows (PowerShell): three terminals

Terminal 1: backend API (from the project folder)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r backend\requirements-dev.txt
cd backend
# DEMO-ONLY setting: the sample data is dated April 2026, so without this every machine reads
"stale"
$env:STALE_AFTER_MINUTES = "100000000"
uvicorn app.main:app --reload
```

The API is now on **http://localhost:8000** and the interactive API docs on **http://localhost:8000/docs**. A background worker runs inside the same process.

Terminal 2: load the supplied seed data (once)

```
.\.venv\Scripts\Activate.ps1
cd backend
python scripts\load_seed.py
```

You should see one line per batch, for example `pulseforge PLANT_01: accepted=5 duplicates=0 batch=…`, for all four sources.

Terminal 3: dashboard

```
cd frontend
npm install
npm run dev
```

Open **http://localhost:5173**. The dev server forwards API calls to port 8000 automatically.

Option B: Docker Compose

```
docker compose up --build
# in a second terminal, load the seed data into the running stack:
docker compose exec api python scripts/load_seed.py --url http://localhost:8000
```

Dashboard: **http://localhost:8080** · API: **http://localhost:8000** · API docs: **http://localhost:8000/docs**. To turn on API-key protection: `$env:API_KEYS="dev-key"; docker compose up --build`, then enter the key with the key button in the dashboard.

> **Verified with Docker Desktop on Windows (October 2026).** The owner's stack built and ran: API healthy, worker and frontend up, the 8 seed batches (25 records) all `processed`, and the live EQ-001 reading processed correctly (section 8.5). The final version (1.5.0) was then verified on a separate, isolated stack (own ports and volume, so demo data was not touched): the end-to-end script passed **55/55 checks passed** with all processing done by the separate worker container; nginx routed the API and dashboard deep links; data survived restarting the api and worker containers; and the runtime image contains no test tools. To deploy the new version on your stack run `docker compose up -d --build`.

Running the tests

```
.\.venv\Scripts\Activate.ps1
python -m pytest backend -q            # backend
cd frontend
npm run typecheck; npm test            # frontend
```

> **Latest results:** backend `153 passed in 6.32s` · frontend `Test Files 2 passed (2) / Tests 16 passed (16)` · strict TypeScript check: no errors.

End-to-end check of a running system

Start the API on an **empty** database, then run (from `backend`):

```
python scripts\verify_scenarios.py --url http://localhost:8000
```

It loads the seed data and every supplied sample over HTTP, waits for the background worker, and checks the *final* processing results: seed outcome, invalid requests, duplicates, malformed records, partial failures, out-of-order events, conflicting updates, work-order scope, revisions, future timestamps and byte-exact traceability. Latest result (isolated Docker stack): **55/55 checks passed**. It uses only the Python standard library.

8. How to see the output

There are three ways to look at the results: the **dashboard** (easiest), the **interactive API docs**, and **calling the API directly**. All screenshots below were taken from the running system with the supplied seed data loaded.

8.1 Dashboard: Overview

The first page. Five numbers at the top (machines monitored, needing attention, critical, processing queue, dead letters), a bar of machines by status, the lines ranked by how many machines need attention, the critical machines, and data-quality problems (failed, partial, stale). Use the plant buttons to filter.

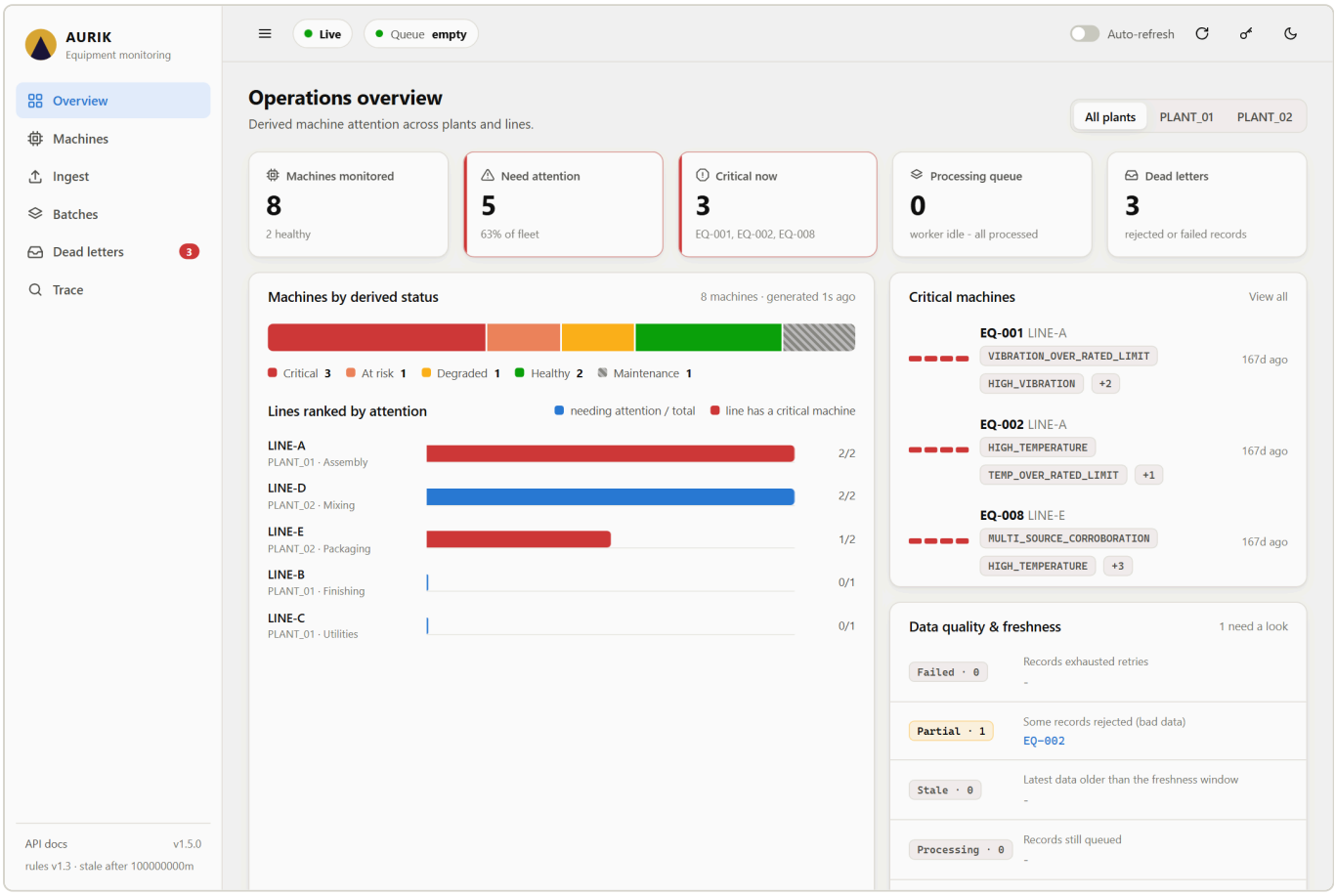


Figure 1. Overview: 8 machines, 5 need attention, 3 critical (EQ-001, EQ-002, EQ-008); LINE-A has the most machines needing attention.

8.2 Dashboard: Machines

One card per machine, most urgent first. Each card shows the status badge, an attention meter (4 bars), the top reason codes and whether the data is fresh. Filter by plant, line, status, "needs attention only", or search by id or reason code.

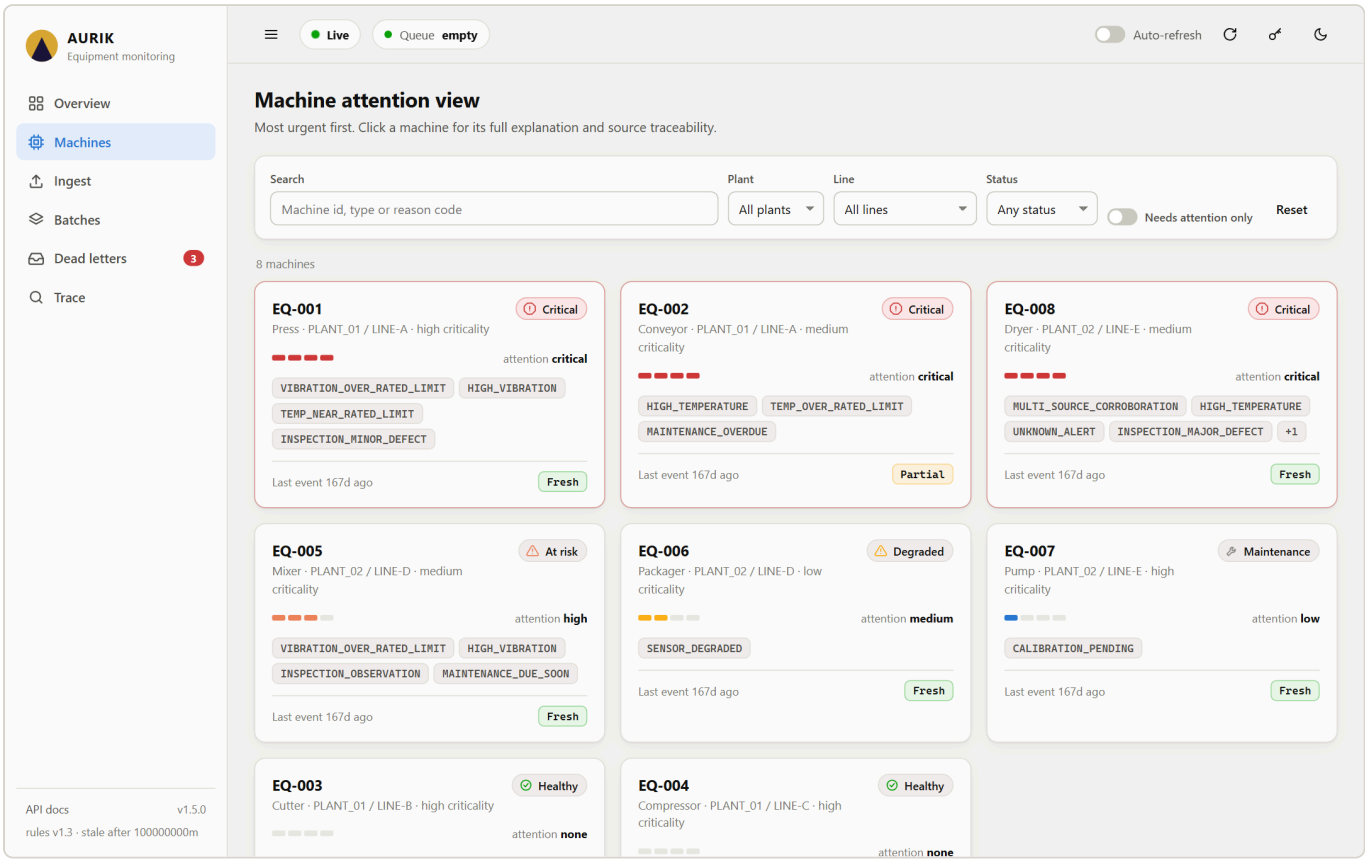


Figure 2. Machines page: cards sorted by attention level.

8.3 Dashboard: machine detail ("why")

Click a card to open its explanation: every reason with its level, the source record (click to trace it), the latest signals from each source, freshness, and version numbers.

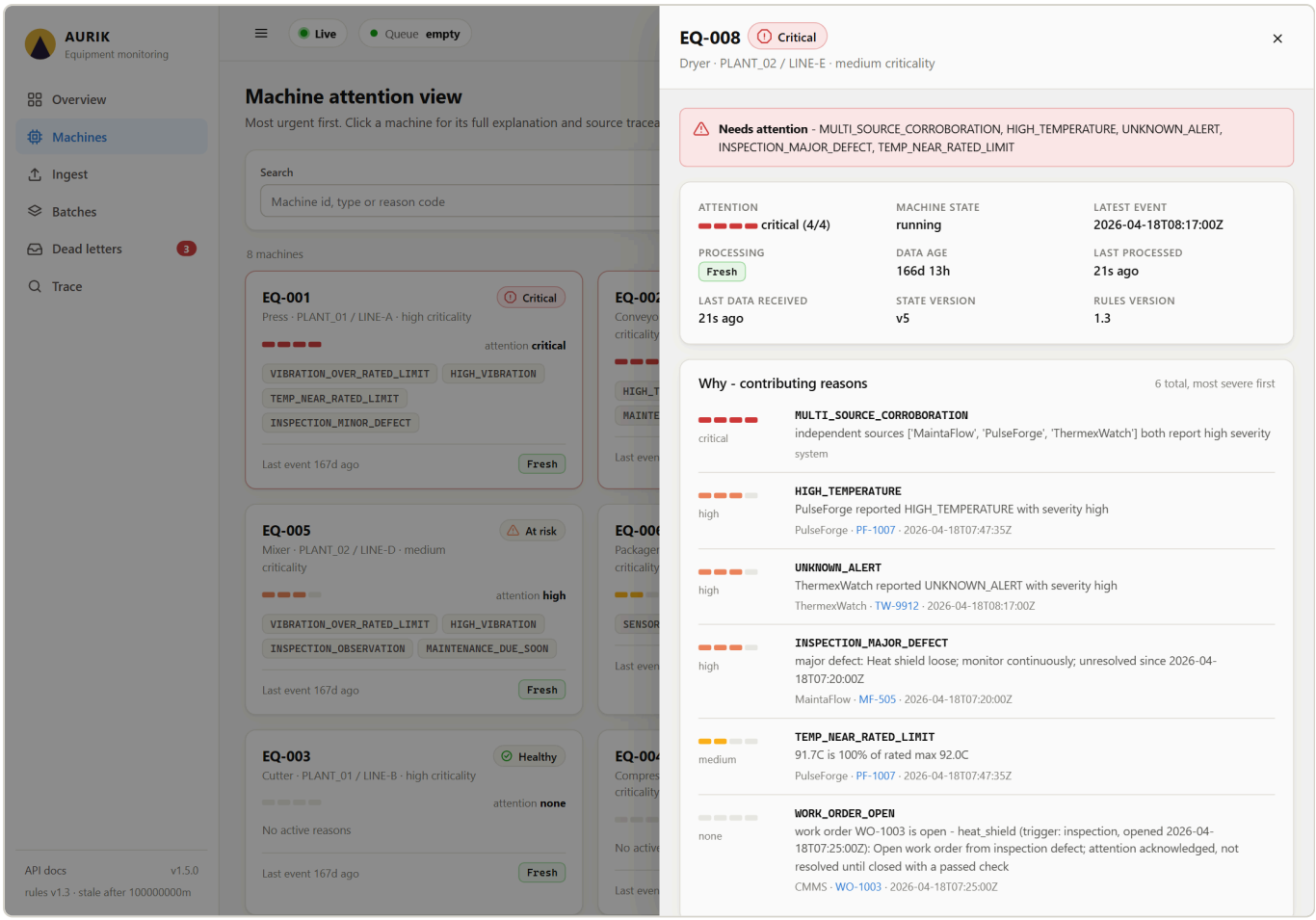


Figure 3. EQ-008 is critical: high temperature from two vendors, a major defect from inspection, raised one level because independent sources agree.

8.4 Example: why EQ-004 is healthy

EQ-004 had power alerts from two vendors at 07:51 and 07:52 and an operator note about noise. Work order WO-1002 was **closed at 08:10 with a passed post-maintenance check** ("Power drop investigated and cleared"). The system marks the earlier power alerts as resolved (a power check covers power), and the operator note too (the work order was triggered by an operator report). It keeps them in the explanation so anyone can see what happened. A vibration or temperature alert would **not** have been cleared by this repair.

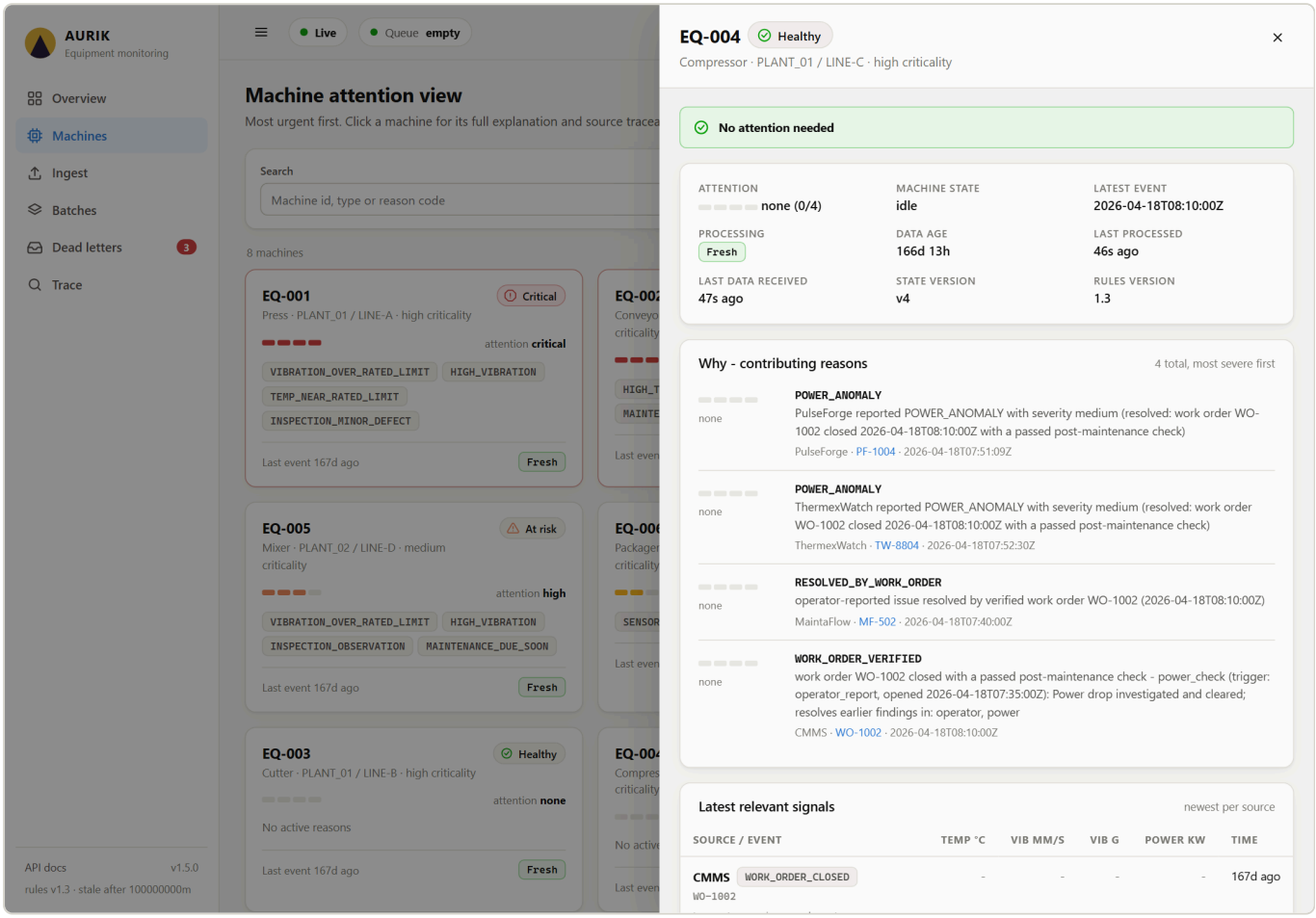


Figure 4. EQ-004: earlier alerts resolved by WO-1002; status Healthy, attention none (0/4).

8.5 Example: EQ-001 with a new reading (the Docker smoke test)

This is the scenario run on the Docker stack: the April seed data plus one live ThermexWatch reading for EQ-001 in October (`TW-DEMO-EQ001-001`: 171 °F, 25.2 kW, vibration warning level 3). The result shows both freshness policies:

- The October reading counts: 171 °F becomes 77.22 °C; the vibration warning becomes `HIGH_VIBRATION` (medium); 25.2 kW is 30% below the 36 kW baseline, so `POWER_DEVIATION` (medium).
- The April PulseForge reading is **ignored** (`STALE_SIGNAL_IGNORED`): it is far older than the newest reading.
- The April MaintaFlow inspection defect (MF-501, belt wear) is **kept** (`INSPECTION_MINOR_DEFECT`, low): nothing has resolved it, and a physical defect does not go away with time. The reason states its age.

Result: **degraded**, attention level 2 (medium), processing status fresh. The same result was observed on the Docker stack and is now an automated regression test.

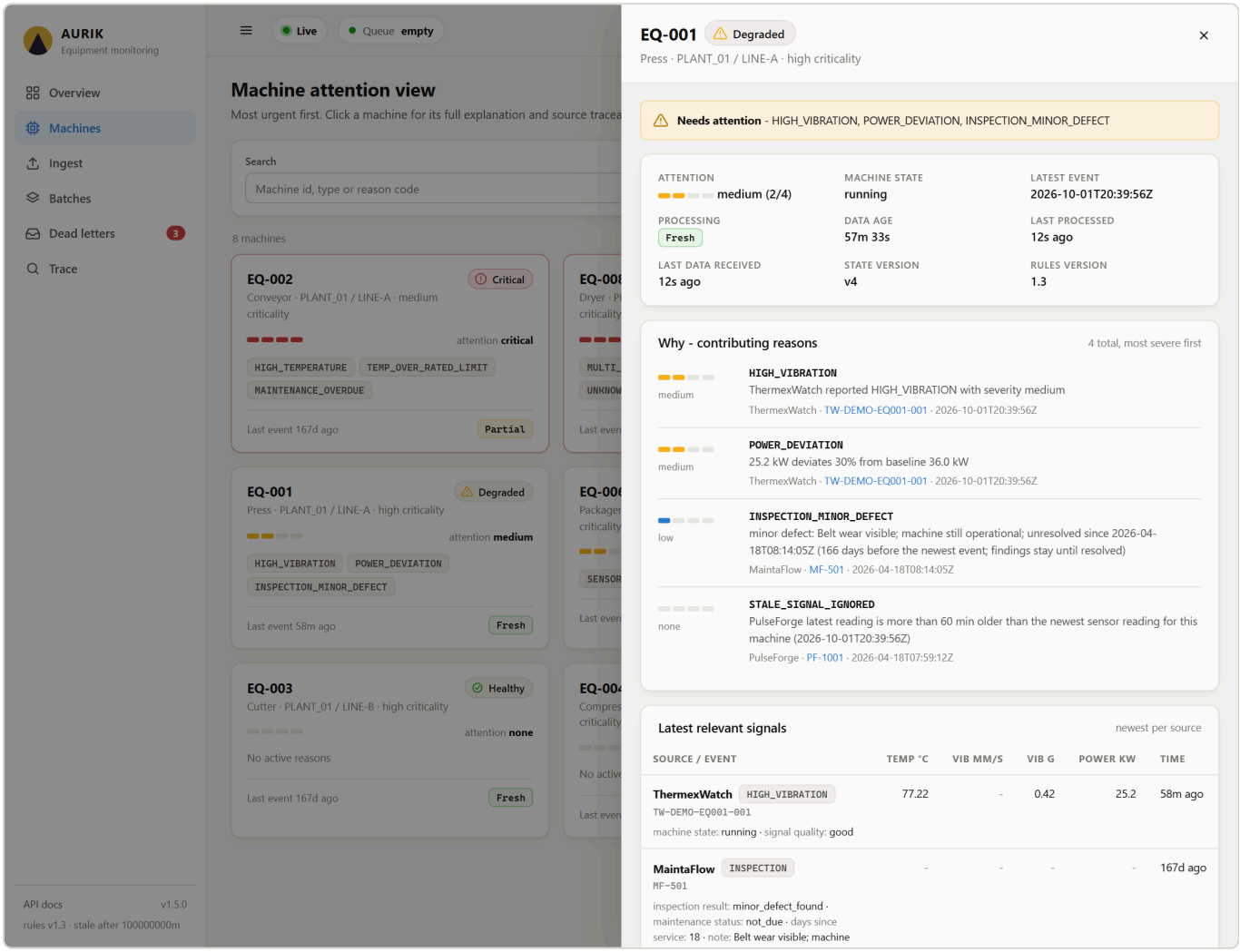


Figure 5. EQ-001 after the October reading: two current reasons from the new reading, one unresolved April finding, the April sensor reading ignored.

8.6 Dashboard: Ingest (try your own data)

Choose a source, paste a batch or pick one of the supplied samples, and the checks on the right update as you type. **Dry-run validate** shows exactly what the server would do with each record; **Submit batch** sends it and shows live processing progress.

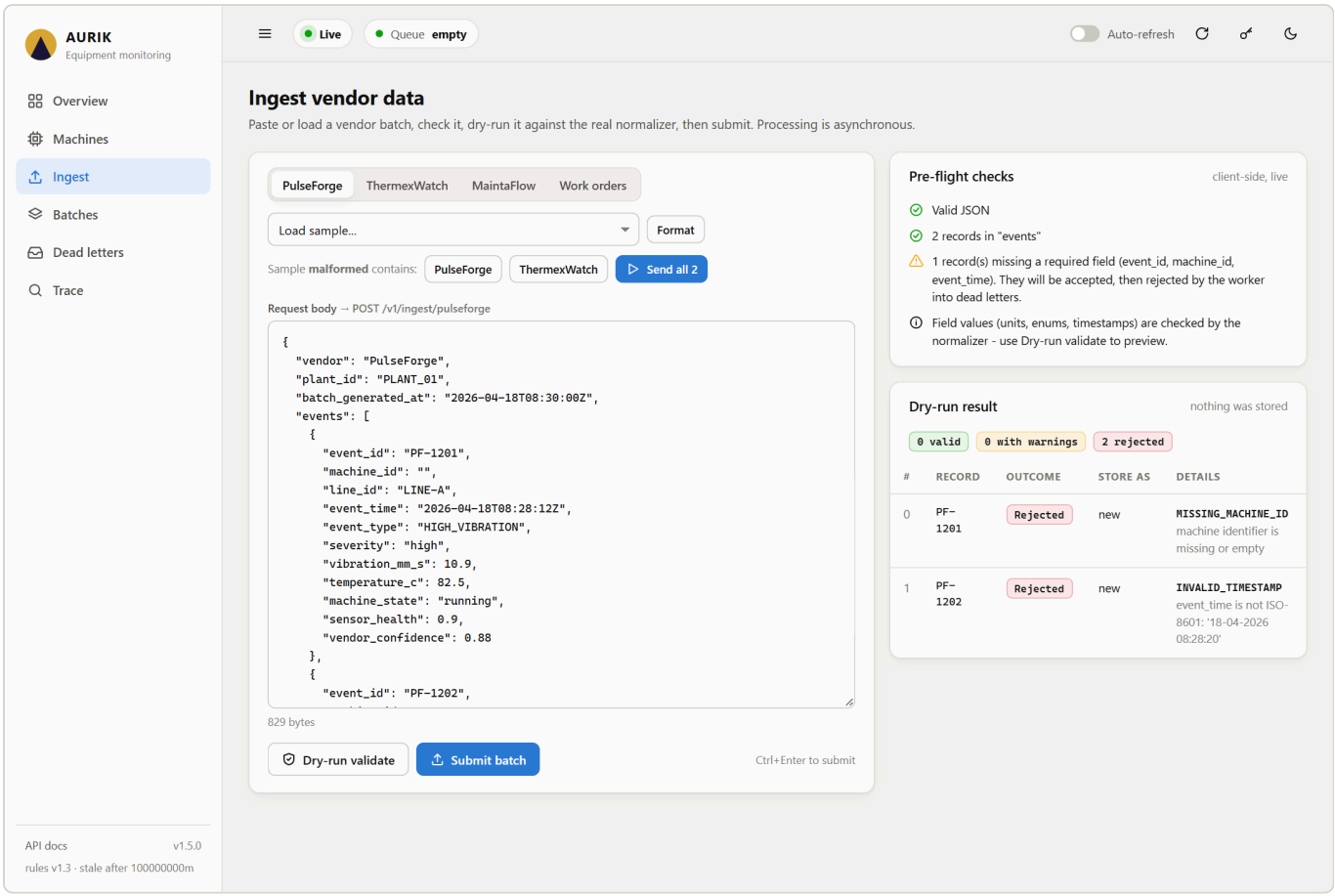


Figure 6. Dry run of the PulseForge section of malformed.json on a database holding only the seed data: both records would be rejected (missing machine id; invalid timestamp) and would be stored as new.

8.7 Dashboard: Batches, Dead letters and Trace

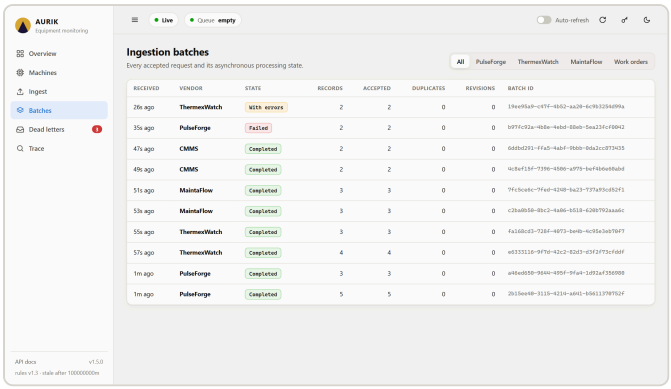


Figure 7. Batches: every accepted request and its processing state.

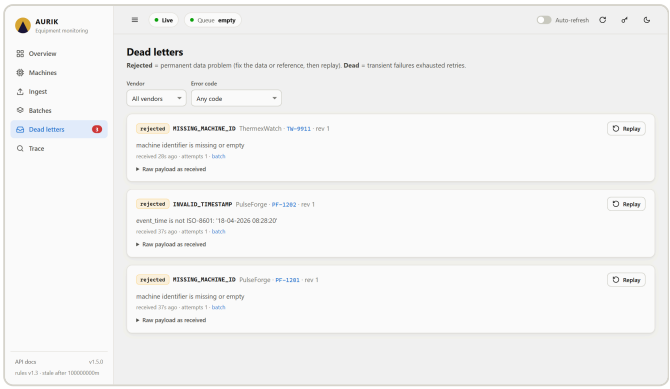


Figure 8. Dead letters: the three rejected records from the two sections of malformed.json.

The Trace view shows the revision rule at work. PF-1001 arrived three times: from the seed CSV (no vendor batch time), from the vendor sample (batch time 08:05) and finally as a delayed copy generated at 07:00. Revision 2 is **authoritative**; revision 3 is kept but marked `superseded` / `STALE_REVISION`, with the reason.

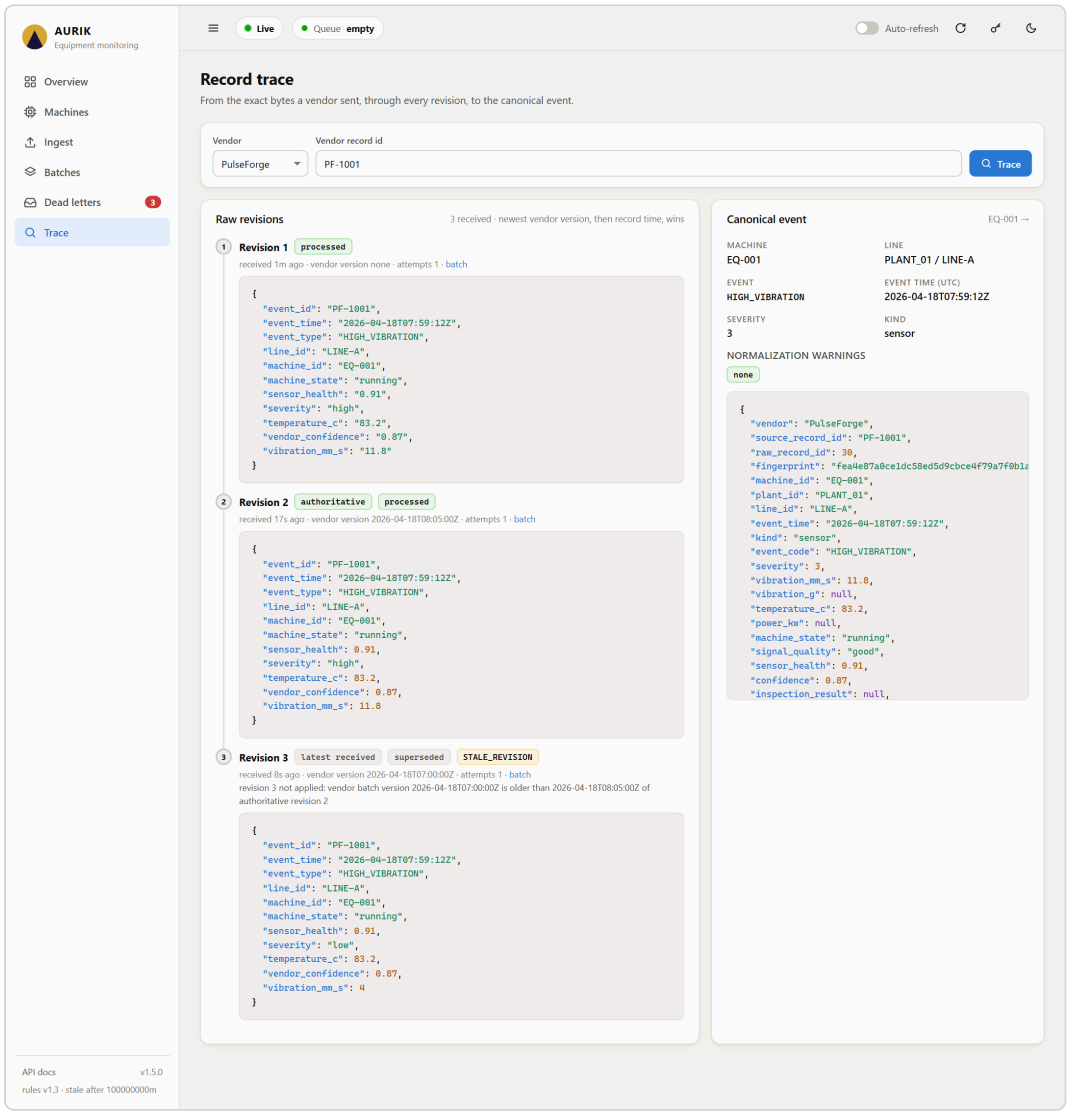


Figure 9. Trace of PF-1001: three raw revisions as received; revision 2 is authoritative, the delayed revision 3 is not applied.

8.8 Interactive API docs (Swagger)

Open **http://localhost:8000/docs**. Every endpoint is listed with its request and response format. Click an endpoint, then **Try it out** → **Execute** to call it from the browser. The same contract is saved in `docs/openapi.json`.

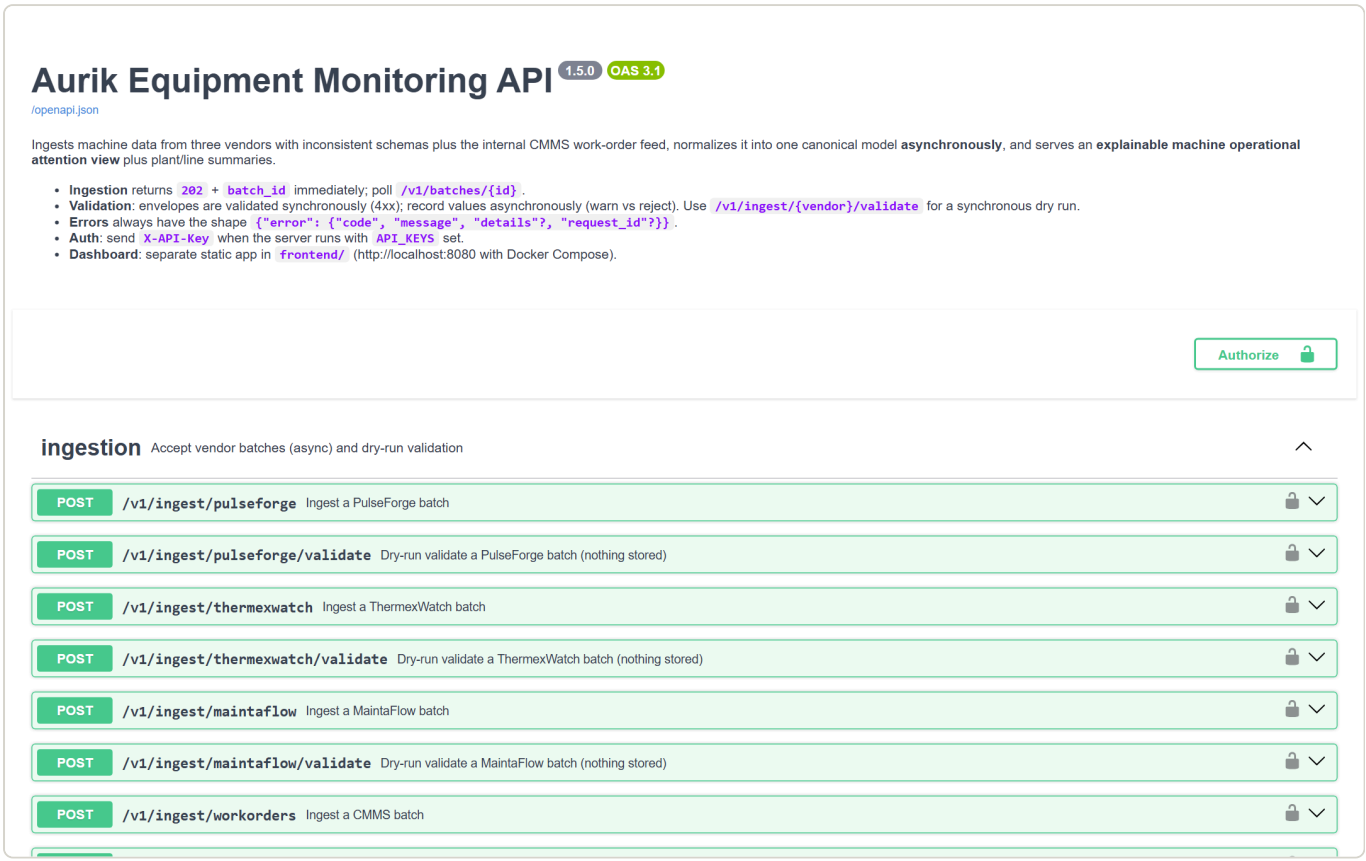


Figure 10. Interactive API documentation generated from the code.

8.9 Calling the API directly (PowerShell)

```
# one machine
Invoke-RestMethod http://localhost:8000/v1/machines/EQ-004 | ConvertTo-Json -Depth 6
# all machines that need attention
Invoke-RestMethod "http://localhost:8000/v1/machines?needs_attention=true"
# plant / line summary (all plants, or one plant)
Invoke-RestMethod http://localhost:8000/v1/summary
Invoke-RestMethod http://localhost:8000/v1/summary/plants/PLANT_01
# send a vendor batch, then check its processing status
$body = Get-Content -Raw backend\sample_payloads\pulseforge_sample.json
$r = Invoke-RestMethod -Method Post -Uri http://localhost:8000/v1/ingest/pulseforge -ContentType
"application/json" -Body $body
Invoke-RestMethod ("http://localhost:8000" + $r.status_url)
# rejected records
Invoke-RestMethod http://localhost:8000/v1/dead-letters
```

Real response: machine view (`GET /v1/machines/EQ-004`, shortened):

```
{
  "machine_id": "EQ-004",
  "plant_id": "PLANT_01",
  "line_id": "LINE-C",
  "derived_status": "healthy",
  "needs_attention": false,
  "attention_level": 0,
  "attention_label": "none",
  "reason_codes": [],
  "reasons": [
    {
      "code": "POWER_ANOMALY",
      "level": 0,
      "source": "PulseForge",
      "record_id": "PF-1004",
      "detail": "PulseForge reported POWER_ANOMALY with severity medium (resolved: work order WO-
1002 closed 2026-04-18T08:10:00Z with a passed post-maintenance check)"
    },
    {
      "code": "POWER_ANOMALY",
      "level": 0,
      "source": "ThermexWatch",
      "record_id": "TW-8804",
      "detail": "ThermexWatch reported POWER_ANOMALY with severity medium (resolved: work order WO-
1002 closed 2026-04-18T08:10:00Z with a passed post-maintenance check)"
    },
    "..."
  ],
  "latest_relevant_event_time": "2026-04-18T08:10:00Z",
  "source_event_refs": [
    {
      "source": "CMMS",
      "record_id": "WO-1002"
    },
    {
      "source": "MaintaFlow",
      "record_id": "MF-502"
    },
    {
      "source": "PulseForge",
      "record_id": "PF-1004"
    },
    "..."
  ],
  "last_processed_at": "2026-10-01T21:36:01Z",
  "rules_version": "1.3",
  "processing_status": {
    "status": "fresh",
    "pending_records": 0,
    "failed_records": 0,
    "rejected_records": 0
  }
}
```

Real response: summary (`GET /v1/summary`, shortened). It answers the four questions in `expected_output_context.md`: machines per status, lines with the most machines needing attention, critical machines, and stale/failed/partial machines.


```
{
  "machines_total": 8,
  "machines_needing_attention": 5,
  "by_status": {
    "critical": 3,
    "at_risk": 1,
    "degraded": 1,
    "healthy": 2,
    "under_maintenance": 1
  },
  "lines_ranked_by_attention": [
    {
      "line_id": "LINE-A",
      "line_name": "Assembly",
      "machines_total": 2,
      "machines_needing_attention": 2,
      "critical_machines": 2
    },
    {
      "line_id": "LINE-D",
      "line_name": "Mixing",
      "machines_total": 2,
      "machines_needing_attention": 2,
      "critical_machines": 0
    },
    "..."
  ],
  "critical_machines": [
    "EQ-001",
    "EQ-002",
    "EQ-008"
  ],
  "data_quality": {
    "stale_machines": [],
    "failed_machines": [],
    "partial_machines": [
      "EQ-002"
    ],
    "pending_machines": [],
    "no_data_machines": []
  }
}
```


Real responses: batch processing status (`GET /v1/batches/{id}`) for the two sections of `malformed.json`, which are sent as two separate batches:

PulseForge section: both records rejected, batch `failed`

ThermexWatch section: one processed, one rejected, batch `completed_with_errors`

```
{
  "vendor": "PulseForge",
  "state": "failed",
  "total_records": 2,
  "accepted_records": 2,
  "record_status_counts": {
    "rejected": 2
  },
  "errors": [
    {
      "vendor_record_id": "PF-1201",
      "status": "rejected",
      "error_code": "MISSING_MACHINE_ID",
      "error_message": "machine identifier is
missing or empty"
    },
    {
      "vendor_record_id": "PF-1202",
      "status": "rejected",
      "error_code": "INVALID_TIMESTAMP",
      "error_message": "event_time is not ISO-
8601: '18-04-2026 08:28:20'"
    }
  ]
}
```

```
{
  "vendor": "ThermexWatch",
  "state": "completed_with_errors",
  "total_records": 2,
  "accepted_records": 2,
  "record_status_counts": {
    "processed": 1,
    "rejected": 1
  },
  "errors": [
    {
      "vendor_record_id": "TW-9911",
      "status": "rejected",
      "error_code": "MISSING_MACHINE_ID",
      "error_message": "machine identifier is
missing or empty"
    }
  ]
}
```


Main endpoints

Endpoint	What it returns
`POST /v1/ingest/{source}`	Accepts a batch → `202` + `batch_id` + `status_url`
`POST /v1/ingest/{source}/validate`	Dry run per record; stores nothing
`GET /v1/batches`, `/v1/batches/{id}`, `/raw`	Processing status, per-record errors, the original request body
`GET /v1/machines`, `/v1/machines/{id}`	Machine attention view (filters: plant, line, status, needs attention)
`GET /v1/summary`, `/v1/summary/plants/{id}`	Plant / line summary
`GET /v1/dead-letters`, `POST …/{raw_id}/replay`	Rejected or failed records; re-queue one
`GET /v1/records/{source}/{record_id}`	Full trace of one vendor record
`GET /health`, `/ready`, `/v1/meta`	Liveness, readiness (queue depth, dead letters), versions

9. Troubleshooting

Problem	Fix
Every machine shows **Stale**	The sample data is from April 2026. Start the API with `$env:STALE_AFTER_MINUTES="100000000"` (section 7).
`Activate.ps1 cannot be loaded`	Run `Set-ExecutionPolicy -Scope Process Bypass` in that terminal, then activate again.
Port 8000 or 5173 already in use	Stop the other process, or use `uvicorn app.main:app --port 8001` and `$env:VITE_PROXY_TARGET="http://localhost:8001"; npm run dev -- --port 5174`.
Dashboard says **Offline · API unreachable**	The backend is not running, or runs on a different port than the dev server proxies to (see above).
`401 UNAUTHORIZED`	The server was started with `API_KEYS`. Click the key button in the dashboard, or send the `X-API-Key` header.
`415 UNSUPPORTED_MEDIA_TYPE` when posting	Add the header `Content-Type: application/json` (in PowerShell: `-ContentType "application/json"`).
Machines stay at **Processing**	No worker is running. Locally it runs inside the API process; with Docker the `worker` container must be up.
Want a clean start	Stop the API and delete `backend\data\monitoring.db*`; it is recreated on the next start.

10. Known limitations

- **Docker:** the final version was verified on an isolated stack; your running demo stack still runs the older version until you redeploy it.
- **Work-order scope** is a mapping of maintenance types to finding categories (my assumption); a CMMS listing the affected components would make it exact.
- **Rule thresholds** (rated-limit ratios, the 60-minute relevance window, the 10-minute future tolerance) are documented assumptions, not vendor facts.
- **Revisions without a vendor batch time** (MaintaFlow, work orders): a correction that moves the record's own time backwards is treated as stale (it stays visible in the trace).
- The same values spelled differently (`"11.8"` in a CSV export vs `11.8` in JSON) count as a new revision; the canonical event is identical.
- **ThermexWatch vibration** stays in g and is not compared with the mm/s rated limits.
- **SQLite + table-as-queue** suits a single host. The production path (Postgres with `SKIP LOCKED` or SQS, metrics, alerting) is described in the README.

Further reading in the repository: `README.md`, `docs/architecture.md` (diagram), `docs/design-note.md` (decisions and trade-offs), `docs/requirements-coverage.md` (requirement → code → test), `docs/openapi.json` (API contract).